UNIFOR Quest — Game Design Document

Título do jogo: UNIFOR Quest

Integrantes:

- Carlos Vinicius
- Daniel Luccio
- Guilherme Abrunheiro
- Henrique

1. Visão geral

Gênero: Ação, Aventura, Puzzle

Plataforma(s): Desktop — Windows (primária), Linux e macOS, via Electron (ADR 0001, ADR 0025).

Público-alvo: 16+ — estudantes de computação, do ensino médio técnico à graduação. O jogo pede que o jogador programe de verdade.

Resumo do conceito: Um jogo de aventura e exploração em 2D ambientado no campus da Universidade de Fortaleza (UNIFOR), no qual o jogador assume o papel de um estudante que percorre a universidade, interage com personagens e resolve desafios para avançar. O diferencial está na integração entre exploração e programação: determinadas missões exigem que o jogador desenvolva soluções utilizando linguagens reais — hoje, TypeScript, Go, Java, Python e Elixir — com os resultados do código influenciando diretamente a progressão no mundo do jogo. Adicionar uma linguagem custa um adapter, nunca reescrever quests (ADR 0003). Dessa forma, conceitos de programação, algoritmos e estruturas de dados são transformados em elementos de gameplay, criando uma experiência que combina entretenimento, exploração e aprendizado.

Visão artística: 2D em perspectiva top-down e pixel art, inspirado em jogos clássicos de exploração e aventura. O campus da UNIFOR é representado de forma estilizada, com cores vibrantes, personagens e cenários de formas simples e facilmente reconhecíveis — atmosfera leve, descontraída, universitária, acessível e divertida (visão artística completa).

2. Mecânicas de jogo

- **Jogabilidade central:**
  - O jogador controla um personagem em visão 2D top-down e explora o campus da UNIFOR, conversa com NPCs, recebe missões e interage com o ambiente.
  - As missões principais são desafios de programação. O jogador tem um editor de código de verdade dentro do jogo — toolchain, servidor de linguagem e projeto reais, não um juiz online (ADR 0046) — e escreve a solução para o problema apresentado pelo NPC. O código

roda nativamente e é avaliado por casos de teste. Resolver corretamente desbloqueia novas áreas, missões ou interações no campus.
    - A exploração também serve para encontrar personagens, objetos e informações necessárias pra resolver certos desafios — programação e exploração ficam integradas, não são duas camadas separadas.

- **Objetivos e progressão:**
    - O objetivo principal é completar as quatro missões da história, em até 20 minutos (ADR 0064). As missões têm dificuldade crescente: strings, árvore binária de busca, fila e, no fim, grafos com Dijkstra — ver progressão e a tabela de quests.
    - Cada lugar do campus só abre depois que a missão do lugar anterior é resolvida.
- **Sistema de recompensas:**
    - Concluir uma missão abre o próximo lugar, a próxima conversa e muda o mundo.
    - O próprio **retorno do código altera o mundo** (ADR 0013).
    - O jogo mostra o desempenho do jogador nos desafios — quantos testes passaram e a **eficiência da solução, medida em contagem de operações, não em tempo de execução** (ADR 0011): tempo de execução é ruído de startup de processo, não um sinal confiável de complexidade.
- **Aplicação de conceitos de computação:**
    - Os conceitos de computação são as próprias mecânicas do jogo, não elementos teóricos à parte.
        - **Grafos:** a rede do campus é um grafo com pesos, e o confronto final é um caminho mínimo com Dijkstra.
        - **Estrutura de dados avançada:** o acervo da Biblioteca é uma árvore binária de busca. O desafio devolve o caminho até o livro, então só passa quem constrói a árvore de verdade.
        - **Matemática:** a cifra de César da portaria é aritmética modular (`(letra + deslocamento) mod 26`), e o confronto soma distâncias com peso.
        - Outras estruturas: strings e fila (o RU).
        - Análise de desempenho: comparar soluções diferentes pro mesmo problema, mostrando complexidade e eficiência de algoritmos.

3. Mundo e narrativa

- **Cenário:**
    - O jogo se passa dentro do campus da UNIFOR, representado de forma estilizada em pixel art: jardins, blocos de aula, biblioteca, Centro de Convivência (o RU, como é chamado na UNIFOR), área esportiva e calçadas arborizadas.
    - A ação acontece numa única manhã, o primeiro dia de aula do protagonista.
    - O jogo inteiro dura **até 20 minutos**, do menu ao final (ADR 0064).

- **História e personagens:** (desenvolvido em docs/story.md; resumo aqui)
  - **Premissa:** no primeiro dia de aula, um vírus que assina como NULL toma o sistema do campus: as telas apagam, as portas travam e os serviços param. O calouro sabe programar, então conserta o campus lugar por lugar até chegar ao esconderijo do NULL e derrotá-lo.
  - **A jornada:**
    1. **Portaria:** as telas mostram `NULL`. Davi tenta ajudar e piora tudo, e Marcos não deixa ninguém passar. Resolver o desafio libera a entrada no campus (ADR 0061).
    2. **Biblioteca:** o NULL derrubou a árvore do acervo de Dona Célia e fala com o jogador pela primeira vez. O caminho até o livro aponta para o Centro de Convivência.
    3. **Centro de Convivência:** o RU travou. Seu Raimundo conta o boato das luzes do Bloco de Tecnologia, que nunca apagam. É lá que o NULL está.
    4. **Bloco de Tecnologia:** o duelo final. O jogador vence o NULL com uma solução mais eficiente que a força bruta dele. As telas do campus voltam, e Davi aparece perguntando o que perdeu.
  - **Tom:** leve e universitário, com um vilão que provoca o jogador pela tela.
  - **Personagens e arcos:**
    - **Protagonista (o calouro):** estudante de Computação no primeiro dia. A história dele é contada pelo código que escreve. **Arco:** chega sem saber nem onde fica a sala de aula e sai como quem salvou o campus.
    - **Davi:** colega de turma e alívio cômico. **Arco:** é quem piora tudo na portaria, e no fim aparece sem ter visto nada, perguntando o que perdeu.
    - **Marcos:** segurança da portaria, desconfiado de tecnologia. Primeiro NPC do jogo, dentro de um cenário próprio (ADR 0059). **Arco:** não deixa ninguém passar e acaba abrindo a portaria graças ao código do calouro.
    - **Dona Célia:** bibliotecária rigorosa. **Arco:** perde o acervo que organizou a vida inteira e, com ele de volta, é quem acha o primeiro rastro do NULL.
    - **Seu Raimundo:** do Centro de Convivência (RU), conhece todo mundo no campus. **Arco:** passa a cena reclamando da fila e termina dando a pista que leva ao NULL.
    - **NULL (antagonista):** o vírus. Sem rosto, fala em texto glitchado e provoca o jogador a cada lugar. **Arco:** do deboche na Biblioteca à derrota no Bloco de Tecnologia.
- **Lore:**
  - **O Fantasma do Bloco de Tecnologia:** o boato de campus sobre as luzes que nunca apagam. São as máquinas do NULL.

4. Níveis e ambientes

- **Estrutura dos níveis:** quatro lugares em sequência, uma quest em cada. Cada lugar só abre depois que o anterior é resolvido:

#	Lugar	Quest	Conceito
1	Portaria	Placas embaralhadas (cifra de César)	Strings e aritmética modular
2	Biblioteca	A árvore do acervo	Árvore binária de busca
3	Centro de Convivência	Fila do Centro de Convivência	Fila
4	Bloco de Tecnologia	O confronto (duelo de operações)	Grafos, Dijkstra e eficiência

    - Cada lugar segue a mesma estrutura: chegada → NPC explica o problema → desafio de código → consequência no mundo → próximo lugar.
    - **Bloqueio e liberdade:** a portaria só deixa sair depois do primeiro desafio. Nos lugares já abertos, o jogador circula livremente, e toda quest roda em qualquer linguagem, com replay livre (ADR 0030).
- **Design dos ambientes:** cada lugar tem ao menos um NPC ou terminal que abre o editor de código.
    - **Portaria:** pequeno interior onde o jogo começa, com Marcos, Davi e o painel de placas.
    - **Campus:** área aberta que liga os prédios, acessada a partir da portaria.
    - **Biblioteca:** estantes altas, mesas de estudo, luz quente das janelas. Com o NULL, livros espalhados e um terminal com a mensagem dele.
    - **Centro de Convivência / RU:** balcão, mesas compridas, uma fila caótica.
    - **Bloco de Tecnologia:** corredores frios, racks piscando, mais escuro que o resto do campus, sem ninguém além do NULL. Arena do confronto.
- **Desafios e obstáculos:**
    - Desafios de programação: implementar uma função a partir da descrição do NPC. No confronto, vencer o NULL com menos operações que a força bruta dele.
    - **Curva de dificuldade:** cada lugar introduz um conceito novo, de strings até grafos.
    - **Falhar sem punição:** não existe game over. Um teste que falha mostra qual caso quebrou e o valor esperado, sem limite de tentativas.

5. Arte e áudio

- **Estilo visual:** 2D top-down, pixel art, inspirado em jogos clássicos de exploração; campus estilizado, cores vibrantes, personagens e cenários de formas simples; atmosfera leve, descontraída, universitária (vision.md). No início, tilesets livres (CC0) valem até o ciclo principal estar provado.
- **Design de Personagens:**
    - **Protagonista (o calouro):** jovem calouro de Computação, roupa casual de estudante, mochila, expressão determinada.
    - **Davi:** colega de turma do protagonista, boné, mochila surrada, sorriso largo. Alívio cômico, abre e fecha o jogo.

    - **Marcos:** segurança da portaria, uniforme, molho de chaves na cintura. Desconfiado de tecnologia, desarmado.
    - **Dona Célia:** bibliotecária, mulher mais velha, cardigã, óculos pendurados por uma correntinha no pescoço. Rigorosa.
    - **Seu Raimundo:** cozinheiro do Centro de Convivência (RU), avental, bigode, jeito cansado mas caloroso. Conhece todo mundo no campus.
    - **NULL:** vírus sem rosto, silhueta feita de texto verde glitchado e estática.
- **Trilha Sonora e Efeitos Sonoros:** chiptune/lo-fi retrô. Arquivos reais em `assets/audio/` (CC0, curados de bancos maiores — ver `LICENSE.txt` em cada pasta):
    - **Música** (`assets/audio/music/`, faixas de 15 Melodic RPG Chiptunes, CC0). Já tocam: `title_screen` no menu inicial, `town` na portaria e no campus, `the_shrine_of_mysteries` na Biblioteca. Reservadas: `bittersweet_story` pro campus enquanto o NULL está no sistema (`town` volta quando ele cai, ADR 0070), `dungeon` pro Bloco de Tecnologia, `the_evil_one` pra chegada ao confronto, `battle_1` pro duelo de operações.
    - **Efeitos de interface** (`assets/audio/sfx/interface/`, de Interface Sounds, Kenney, CC0): confirmação de teste passado, erro de teste falho, clique e seleção de menu (já tocam), e dois `glitch` pras aparições do NULL.
    - **Efeitos de ambiente** (`assets/audio/sfx/rpg/`, de RPG Audio, Kenney, CC0): passos, abrir/fechar livro (Biblioteca), abrir/fechar porta.
    - **Efeitos do NULL** (`assets/audio/sfx/scifi/`, de Sci-fi Sounds, Kenney, CC0): ruído de computador e um bipe retrô como motivo sonoro do NULL — sem voz.

6. Interface do usuário (UI)

- **Layout e Design:**
    - *Implementado:* dentro de uma quest, o editor Monaco ocupa o centro da tela, com uma barra lateral mostrando o diálogo do NPC e a assinatura da função, uma barra superior com o seletor de linguagem e o status do servidor de linguagem (LSP), e um painel de resultado dos testes abaixo do editor.
    - *Implementado:* uma tela de título antes da portaria, com **Iniciar novo jogo** e **Carregar jogo** (desabilitada até existir save), e pausa pelo Esc em toda cena de gameplay (ADR 0062). Uma HUD no canto da tela mostra o objetivo atual (ADR 0063).
- **Funcionalidade:**
    - *Implementado:* o jogador interage com a UI da quest pelo teclado (digitar código) e mouse (clicar em Executar, trocar de linguagem); os resultados de cada caso de teste aparecem com um ✓ ou ✗ e o valor esperado vs. o que veio, sem precisar reler código.
    - *Planejado:* **Carregar jogo** lê o autosave que já existe por quest concluída (ADR 0020) — não existe save manual, o menu só dá acesso a ele.
- **Acessibilidade:** navegação por teclado (setas + Enter) sempre em paralelo ao mouse, nunca só uma das duas. O indicador de teste passou/falhou já usa ✓/✗ além de verde/vermelho —

seguro pra daltonismo sem precisar mudar nada. Zoom/tamanho de fonte ajustável, suporte a leitor de tela e remapeamento de teclas ficam **fora de escopo** desta entrega — decisão da equipe, não uma omissão.

7. Considerações técnicas

- **Motor de jogo:** não é um motor único — Electron (empacotamento desktop) + Phaser (a cena 2D) + Monaco (o editor) + Vite (build do renderer), em TypeScript strict, Node 24 LTS, npm workspaces, Biome e Vitest. Ver Stack em CLAUDE.md e docs/architecture.md.
- **Otimização:** execução do código do jogador roda no processo principal, atrás de um `Executor` com timeout e kill da árvore de processos inteira (ADR 0009); os runtimes de cada linguagem vêm embutidos no instalador, sem servidor remoto e sem instalação pelo jogador (ADR 0021).

8. Cronograma e orçamento

- **Fases de desenvolvimento** (roadmap completo):
  - **Fase A — a mecânica (concluída):** o ciclo abrir desafio → escrever código → executar → testes → concluir → efeito, de ponta a ponta. Marcos: o domínio (`packages/core`), o executor, os adapters das cinco linguagens (TypeScript, Go, Java, Python, Elixir), a suíte de conformance, o editor com servidor de linguagem e a quest "hello world" rodando nas cinco.
  - **Fase B — o jogo (em andamento):** o conteúdo em cima da mecânica. Marcos já entregues: mapa do campus gerado, portaria e Biblioteca, menu de título, pausa, trilha sonora, HUD de objetivo e a quest da portaria jogável. Marcos que faltam: os interiores do Centro de Convivência e do Bloco de Tecnologia, as quests 2 a 4 ligadas a NPC em cena, arte, save e o instalador.
- **Metodologia:** Kanban. O trabalho corre em fluxo contínuo, sem sprints fixas: cada item do roadmap vira um cartão, e cada entrega é um pull request revisado e validado pela CI (lint, typecheck e testes) antes de entrar na `main`.
- **Equipe:** multidisciplinar, sem papéis fixos. Todos os integrantes (Carlos Vinicius, Daniel Luccio, Guilherme Abrunheiro e Henrique) programam, escrevem conteúdo e revisam; o quadro Kanban distribui as tarefas conforme a disponibilidade de cada um.